VaultGuard

Production Requirements Document (PRD)
End-to-End Encrypted Password Manager

Frontend: Next.js + TypeScript + Tailwind CSS | Database: MongoDB Atlas | Authentication: Email/Password + Optional TOTP 2FA

Document purpose: Define a production-ready architecture, UX/UI system, security model, testing strategy, and implementation plan for a zero-knowledge-style password manager. The backend must not receive plaintext vault contents.

1. Product Overview

Product objective

Build a professional password manager for storing website credentials, secure notes, payment information, Wi-Fi credentials, API keys, recovery codes, and custom secure fields.

Primary security principle

Vault data is encrypted on the client before persistence. The server/database should operate on ciphertext wherever practical. The design must make a database-only compromise insufficient to reveal stored credentials.

Core principles

Security by design; zero-knowledge-style vault architecture; simple UX; accessible responsive UI; strong session security; auditable security events; production-grade testing.

2. Technology Stack

Application

Next.js App Router, TypeScript, Tailwind CSS, React Hooks, reusable feature-oriented architecture.

Backend

Next.js Route Handlers/API endpoints with strict authorization and validation.

Database

MongoDB Atlas with TLS, least-privilege database users, backups, monitoring and appropriate auditing.

Validation

Zod for shared request/form schemas.

Authentication

Secure server-side session with HttpOnly, Secure, SameSite cookies.

Cryptography

Argon2id for password hashing/key derivation where appropriate; AES-256-GCM for vault encryption; cryptographically secure random values from Web Crypto.

Testing

Vitest for unit/integration tests; Playwright for E2E; axe-based accessibility testing; dependency/security scanning.

3. Security & Encryption Architecture

Authentication password

Never store plaintext passwords. Hash authentication passwords with Argon2id and unique salts. Never use reversible encryption or fast hashes such as SHA-256 for password storage.

Vault encryption

Derive a key-encryption key locally from the user's master secret using a memory-hard KDF. Use it to unwrap a randomly generated vault encryption key. Encrypt vault records with authenticated encryption such as AES-256-GCM.

Trust boundary

The server must never receive the master password, plaintext vault encryption key, or plaintext vault entries. Plaintext vault data exists only in the trusted client context while the vault is unlocked.

Encryption metadata

Every encrypted record should carry version information such as cryptographic version, algorithm, KDF parameters, IV/nonce, salt and ciphertext so future migration is possible.

Key rotation

Design versioned keys and a migration path. Re-encrypt data using a new key/version without exposing plaintext to the backend.

Critical rule

Do not implement the vault as plaintext data stored in MongoDB and encrypted/decrypted only on the Next.js server. Encryption must occur before sensitive vault data leaves the trusted client context.

4. Authentication & Account Lifecycle

Registration

Collect name, email, password and confirmation. Validate client and server side, hash authentication password, create vault key material, create encrypted vault metadata and establish a secure session.

Login

Email + password. If TOTP is enabled, require the second factor before establishing a fully authenticated session.

Sessions

Use secure HttpOnly cookies; Secure in production; appropriate SameSite policy; session expiry, rotation, revocation and reauthentication for sensitive actions. Avoid localStorage/sessionStorage for session identifiers.

Password reset

Zero-knowledge architecture creates a deliberate recovery limitation. A normal server-side password reset must not silently grant access to the encrypted vault. Provide a recovery-key mechanism or clearly state that an unrecoverable vault is lost if both the master secret and recovery mechanism are lost.

Account security

Rate limit login, registration, password reset, recovery and OTP verification. Prevent account enumeration and avoid leaking whether an email exists.

5. Optional TOTP 2FA

Setup flow

Security Settings → Enable 2FA → generate secret → show QR code → user scans with an authenticator app → enter 6-digit code → verify → enable.

Storage

Encrypt the TOTP secret using an appropriate server-side protection strategy. Recovery codes should be stored as hashes, not plaintext.

Recovery

Generate one-time recovery codes and require the user to confirm they have securely stored them.

UX

Provide clear enabled/disabled state, reauthentication before disabling, backup/recovery instructions, and rate limiting for OTP attempts.

6. Vault Features

Login items

Name, username, password, URL, notes, tags, favorite.

Secure notes

Encrypted free-form notes.

Payment cards

Card name, holder, number, expiry, CVV/PIN and notes. Mask sensitive values by default.

API keys

Service, environment, API key, secret, endpoint and notes.

Wi-Fi/identity/custom

Support structured secure records and custom fields without assuming the server can inspect their contents.

Password generator

Length control, character sets, ambiguous-character exclusion, strength indicator, regenerate, copy and use-in-item.

Favorites & tags

Favorites plus tags such as Work, Personal, Development, Finance and Cloud.

Search

Search decrypted vault contents locally after unlock. Do not send plaintext search terms or decrypted records to the server.

7. Vault Lock & Clipboard

Auto-lock

Configurable inactivity timeout such as 5, 10, 15 or 30 minutes, with optional lock-on-tab/background behavior.

Manual lock

Prominent Lock Vault action.

Clipboard

Copy sensitive values with an automatic clipboard timeout where feasible. Do not expose copied secrets in logs, URLs or analytics.

Memory

Minimize plaintext lifetime in application state and avoid unnecessary duplication. JavaScript cannot guarantee perfect memory wiping, so present this as defense in depth rather than absolute memory erasure.

8. Database Model

Collections

users, sessions, vaults, vault_items, two_factor, recovery_codes, security_events, devices.

users

Store account identity, email verification state, password hash, timestamps and status. Never store the master password.

vaults

Store user/vault identifiers, encrypted vault key, key version, encryption version and timestamps.

vault_items

Store identifiers plus ciphertext and minimal non-sensitive metadata. Sensitive fields must not be stored as plaintext.

Security events

Record event type, timestamp, user/device context and outcome while excluding secrets, tokens, passwords, TOTP secrets and vault plaintext.

9. UI/UX Design System

Design direction

Modern SaaS + classic security product + premium fintech aesthetic. Calm, trustworthy, minimal and professional. Avoid a 'hacker dashboard' visual language.

Color system

Light: background #F8FAFC, surface #FFFFFF, primary #2563EB, text #0F172A, secondary #64748B, border #E2E8F0. Dark: background #020617, surface #0F172A, primary #3B82F6, text #F8FAFC, secondary #94A3B8, border #1E293B.

Typography

Geist or Inter, with restrained heading sizes and strong hierarchy.

Components

Button, Input, PasswordInput, OTPInput, Modal, Drawer, Toast, Dropdown, Tooltip, Card, Badge, Tabs, Dialog, Command Palette, SearchInput, Sidebar, Navbar, Avatar, SecurityScore, PasswordStrength, VaultCard, VaultItem, EmptyState and ConfirmDialog.

Motion

Use subtle 200–300ms transitions, respect prefers-reduced-motion, and avoid excessive animation in a security-focused product.

10. Main Screens & UX

Landing page

Hero, value proposition, security explanation, CTA to create vault, sign-in CTA and concise trust/security messaging.

Dashboard

Greeting, global vault search, Add Item CTA, security score, category counts, recent/favorite items and quick actions.

Vault

Desktop sidebar + main list/detail view; collapsible sidebar on tablet; bottom navigation on mobile.

Add Item

Modal/drawer with item-type selection: Login, Secure Note, Credit Card, Identity, API Key, Wi-Fi and custom item.

Security Center

Security score, 2FA state, recovery state, weak/reused/old password indicators, sessions and security activity.

Settings

Profile, security, auto-lock, appearance, session management, recovery, preferences and account actions.

Empty/error states

Clear, friendly messages without exposing stack traces, database errors or sensitive implementation details.

11. Accessibility & Responsive Design

Accessibility target

WCAG 2.2 AA.

Requirements

Semantic HTML, keyboard navigation, visible focus states, accessible labels, correct contrast, screen-reader support, accessible dialogs and password visibility controls.

Responsive

Desktop sidebar; tablet collapsible navigation; mobile bottom navigation. All core vault operations must be usable without a desktop-only interaction.

12. Next.js Architecture

Recommended structure

src/app for routes; src/components for shared UI; src/features for auth/vault/security domains; src/lib/crypto for cryptography; src/lib/auth for sessions; src/lib/mongodb for database access; src/lib/validation for Zod schemas; src/hooks; src/types; src/config.

Crypto isolation

Create dedicated modules such as encryption.ts, decryption.ts, key-derivation.ts, vault-key.ts, recovery.ts, random.ts and types.ts. Do not scatter crypto operations across React components.

API surface

POST /api/auth/register, /login, /logout, /2fa/setup, /2fa/verify; GET/POST /api/vault; PUT/DELETE /api/vault/:id; GET/DELETE /api/sessions; GET /api/security/activity.

13. Backend & API Security

Validation

Validate every untrusted input server-side with Zod. Client validation is UX only.

Authorization

Every vault/session/security resource must be scoped to the authenticated user. Prevent IDOR by never trusting client-provided ownership identifiers.

Rate limiting

Apply limits to authentication, OTP, registration, recovery and other sensitive endpoints.

CSRF

For cookie-based authentication use an appropriate CSRF strategy plus SameSite and Origin/Referer protections where applicable.

Security headers

Use CSP, HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy and frame-ancestors. Use no-store for sensitive responses where appropriate.

Secrets

Keep server secrets in environment/secret management. Never expose them via NEXT_PUBLIC_ variables or commit them to Git.

14. XSS, Injection & Privacy Controls

XSS

Use React escaping, strict CSP, safe rendering, trusted dependencies and sanitization for any rich user-generated content.

Sensitive data leakage

Never put passwords, vault plaintext, API keys, card data, recovery keys or tokens into URLs, query parameters, analytics payloads, logs or error messages.

NoSQL injection

Validate and constrain MongoDB query inputs; never construct queries from untrusted arbitrary objects.

Privacy

Avoid unnecessary analytics. If analytics are added, exclude all decrypted vault contents and sensitive fields.

15. Security Center & Password Health

Security score

Display a clear score with explanatory checks: strong master password, 2FA, recovery key, weak/reused/old/compromised passwords, active session review and security activity.

Password health

Analyze the decrypted vault locally for weak, reused and old passwords. Do not upload plaintext passwords for analysis.

Security activity

Show successful/failed login, password change, 2FA changes, new device, vault lock/unlock, recovery and session revocation events.

Device management

List active sessions/devices with last-active information and allow revocation plus 'sign out all other devices'.

16. Threat Model

Database compromise

Client-side vault encryption; database contains ciphertext.

Network interception

HTTPS/TLS, HSTS and secure cookies.

Credential database theft

Argon2id password hashing, unique salts, rate limiting.

Session theft

HttpOnly/Secure/SameSite cookies, rotation, expiry and revocation.

XSS

CSP, safe rendering, dependency controls and minimal plaintext exposure.

Brute force

Rate limiting, progressive delays, 2FA and risk controls.

Lost master secret

Recovery key or explicit unrecoverable-vault policy.

17. Testing Strategy

Unit tests

Test encryption/decryption, key derivation, password generation, validation, strength calculation, TOTP and session helpers.

Crypto invariants

decrypt(encrypt(data)) must equal data; wrong key must fail; modified ciphertext/IV/tag must fail.

Integration tests

Test API authorization, ownership boundaries, sessions, rate limiting, registration/login, 2FA and recovery.

E2E tests

Playwright flows for registration, login, 2FA, vault CRUD, search, lock/unlock, sessions and recovery.

Accessibility

Automated axe checks plus keyboard and screen-reader-oriented acceptance tests.

Security tests

NoSQL injection, XSS, CSRF, SSRF, IDOR, authentication bypass, session fixation/hijacking, brute force, credential stuffing, account enumeration, OTP brute force, replay, rate-limit bypass and dependency vulnerabilities.

Compromise test

Assume an attacker has read-only access to MongoDB. Verify that stored credentials remain unrecoverable without the client-side cryptographic secrets.

18. Production Checklist

Authentication

Strong password hashing; secure sessions; TOTP; recovery mechanism; rate limiting; secure account lifecycle.

Encryption

AES-256-GCM; strong KDF; unique salt/nonce; key versioning; migration/rotation plan; no plaintext vault on server.

Database

Atlas TLS; least privilege; restricted access; encrypted backups; monitoring and appropriate audit logging.

Frontend

CSP; XSS defenses; no client-bundled secrets; no sensitive localStorage; clipboard timeout; auto-lock.

Backend

Input validation; authorization; rate limiting; CSRF protection; secure cookies; security headers; no sensitive logs.

Operations

Dependency scanning, secret scanning, CI checks, error monitoring, backups, disaster recovery documentation and security incident procedures.

19. Definition of Done

Functional

Registration, login, logout, TOTP, recovery, vault CRUD, local search, password generator, favorites, tags, security center, sessions and responsive UI all work.

Security

Vault data is encrypted before persistence; MongoDB compromise does not reveal stored passwords; secure cookies; rate limits; authorization; CSP; no sensitive logging; Atlas TLS and least privilege.

Quality

Unit, integration, E2E, accessibility, security and performance tests pass. No critical/high dependency vulnerabilities remain without documented exception.

Documentation

Threat model, recovery behavior, encryption versioning, deployment security and operational runbooks are documented.

20. Antigravity Implementation Instructions

Development sequence

Phase 1: foundation and design system. Phase 2: authentication/session security. Phase 3: cryptographic layer. Phase 4: vault CRUD. Phase 5: TOTP/recovery. Phase 6: security center/sessions. Phase 7: responsive UX/accessibility. Phase 8: automated testing/security tests. Phase 9: production hardening and deployment.

Coding standards

Use TypeScript strict mode, feature-oriented modules, reusable components, server-side validation, explicit error handling, secure defaults and minimal dependencies. Do not duplicate business logic.

Security-first acceptance

Do not mark a feature complete if it compromises the trust boundary. Any feature that requires the server to inspect plaintext vault data must be rejected or redesigned.

Marketing language

Do not claim 'unhackable' or '100% secure'. Describe the actual threat model, encryption architecture, recovery limitations and security controls.